

高质量微调数据工程与评估



>> 今天的学习目标

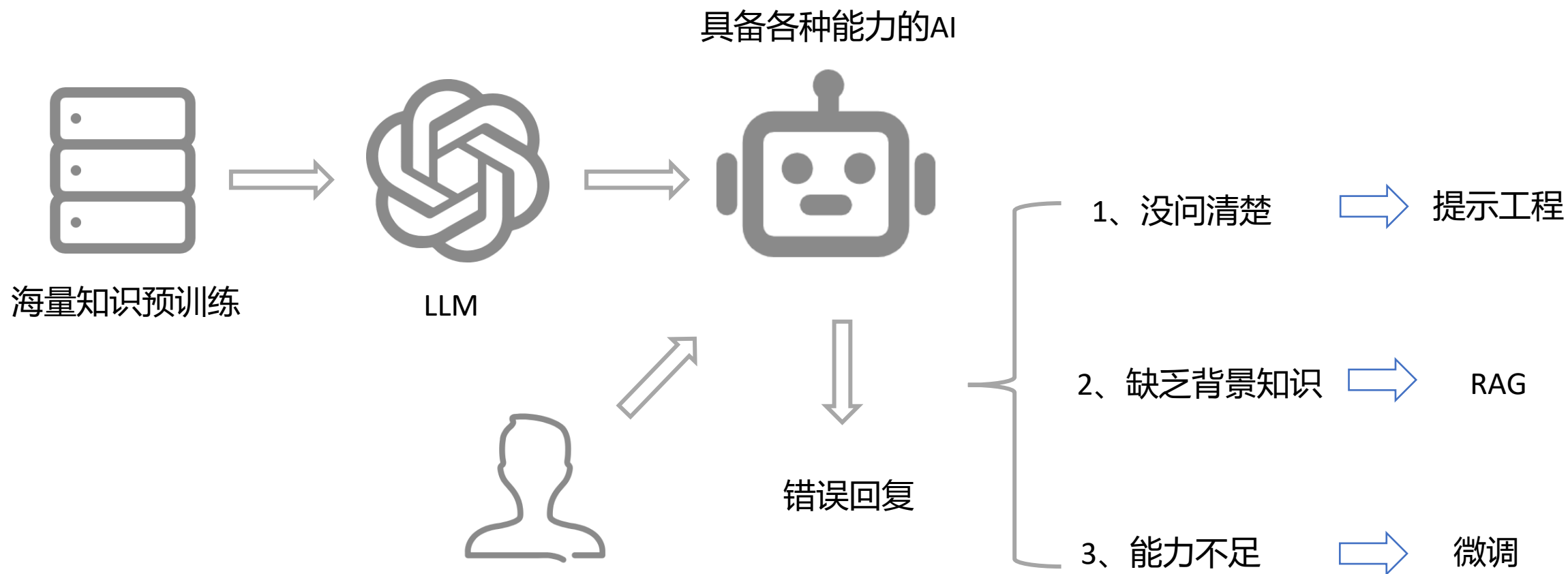
高质量微调数据工程与评估

- 微调方法总览
- CASE: 金融垂类大模型（智能客服）
- Unsloth: LLM高效微调
- CASE: 训练垂类模型（中文医疗模型）
- 搭建模型微调Pipeline
- 数据质量评估体系

微调方法总览

大模型应用开发

Thinking: 提示工程 VS RAG VS 微调, 什么时候使用?



模型的学习方式

Thinking, 不同微调方法对数据的要求完全不同, 都有哪些模型的学习方式?

- SFT (监督微调)

就像老师手把手教学生背诵标准答案。模型看到的是完整的"问题-标准答案"对, 通过模仿来学习。这是最基础的微调方式, 模型直接记忆正确的回答格式和内容。

例子: 训练电商客服机器人时, 我们可以提供大量这样的数据:

输入: "我要退货, 怎么操作?"

输出: "亲, 退货流程如下: 1) 进入订单详情页; 2) 点击申请退货; 3) 选择退货原因; 4) 提交后等待审核..."

模型学习的是: 当用户问退货相关问题时, 应该按照这个固定格式和步骤回答。重点是答案必须准确、格式规范, 不能教错知识。

模型的学习方式

- RLHF (基于人类反馈的强化学习)

就像老师让学生写作文，然后打分排名。模型先尝试生成多个不同回答，人类标注员对这些回答进行排序（比如A比B好），模型通过奖励模型学习人类的偏好，逐步调整生成策略。

例子：训练一个创意写作助手，针对提示"写一段关于秋天的文案"：

模型生成3个版本：A（文艺优美）、B（干巴巴）、C（跑题了）

人工标注：A > B > C

=> 奖励模型学会文艺性和相关性是重要指标，后续生成会偏向这类风格

关键是需要大量人工标注的偏好数据，成本较高，但能教会模型什么是好回答。

模型的学习方式

- GRPO (组相对策略优化)

这是DeepSeek-R1采用的新方法，就像学生自己组队刷题，互相比谁的解题思路更好。不需要额外训练奖励老师（奖励模型），而是让模型针对同一道题生成多组回答，通过组内比较（看谁的答案对/思路好）来自动计算奖励。

例子：训练数学推理模型解 鸡兔同笼 问题：

模型针对同一题生成4个不同的推理过程（组）

其中2个得到正确答案，2个错误

系统自动给正确的2个高奖励，错误的低奖励

=> 模型学会：分步验证数量关系 的推理路径比 瞎猜 更可靠

不同微调方法的数据要求

不同微调方法对数据的要求完全不同：

SFT（监督微调）的数据要求：

- 数据格式：指令-回答对（Alpaca格式）

```
{"instruction": "...", "input": "问题", "output": "标准答案"}
```

=> 有明确的标准答案，模型直接学习"输入->输出"的映射

=> 教模型学会格式和知识

- 典型数据集：Alpaca-52K、中文医疗对话数据集

Thinking：数据质量关键点是什么？

答案的准确性、完整性、格式规范性

不同微调方法的数据要求

RLHF/GRPO (强化学习) 的数据要求:

- 数据格式: 只需要QA对 (如GSM8K), 不需要reasoning标注

```
{"question": "数学题", "answer": "42"}
```

=> 没有推理过程标注, 模型通过奖励函数自己学习推理策略

=> 教模型学会推理和偏好

- 典型数据集: GSM8K (数学推理)

Thinking: 数据质量关键点是什么?

问题的多样性、答案的可验证性

如何选择微调方法

Thinking: 如何选择微调方法?

- 有标准答案的任务（客服、医疗问答） -> SFT
- 需要推理能力的任务（数学、编程） -> GRPO

Thinking: 实际项目中能否两者结合?

可以的, SFT先学格式和知识 -> GRPO再学推理策略

本节课聚焦SFT数据的工程实践, 下节课会涉及GRPO

金融垂类大模型（智能客服）

大模型应用开发 Q&A

大模型应用场景：

1) 对话大模型

主要应用于电话销售、催收、客服场景

2) 文档问答大模型

用于知识中台、在线客服的辅助场景。

知识中台：针对在不同时期文档中相同问题不一致、甚至矛盾的答案的寻找和处理。

客服智能辅助：文档问答大模型+传统智能辅助+大模型对话形成一个综合的客服人员智能辅助方案。

3) SQL生成大模型

解决数据分析效率问题，很多金融机构内部有数十万张表

Step1, 找到相关的数据表；Step2, 基于需求撰写SQL

大模型应用开发 Q&A



传统人工电销

- 合规性差，投诉及监管风险高
- 致电效率低
- 坐席能力参差不齐
- 坐席招聘、培训、管理成本高，人员流动性大



传统智能电销

- 灵活性差：话术配置能应付的场景有限、对客户的异议问题应答能力不足，不够人性化
- 配置工作繁杂：对于每一个细分的外呼业务，都需要配置一套庞大的话术剧本，需要大量人力劳动



大模型应用

- 大幅节省人力资源和时间成本
- 通过海量合规话术的训练，确保外呼过程符合合规要求
- 大大减少了传统人工外呼中的客户意图理解错误和信息错误
- 根据客户的特征和历史记录，提供个性化的推荐和服务

金融大模型的挑战

大模型在金融对客场景中的挑战：

1) 数据质量要求高

直接对客需要大模型的准确性很高，其中数据质量是关键

2) 金融营销业务场景复杂

比如营销分的计算涉及首贷、复贷、无余额，而且各场景还有自己的活动专项和特殊优惠、临时活动等

3) 金融营销场景对信息的准确性要求极高。

用户信息幻觉问题对客户服务质量影响十分明显，比如用户的姓名和年龄要保证 100%准确性，否则会极大影响营销效果

4) 金融营销的安全合规、客户隐私保护等要求极高

5) 金融营销手段千变万化，营销策略日新月异，大模型需要敏捷迭代



营销外呼大模型训练 +
Prompt Engineering

营销外呼大模型训练

垂直场景的大模型生产主要分为两个模块：数据收集、大模型训练

• **数据收集**（以马消为例，已有1.8亿客户，营销外呼量为 百万/天，积累了海量高质量数据）

- ✓ 历史业务数据：优秀坐席服务语音对话、文本对话
- ✓ 业务文档数据：涉及贷前、贷中、贷后等超数万篇服务文档
- ✓ 业务规则数据：客服机器人配置规则和决策树及各个节点的话术
- ✓ 技术相关数据：所有SQL代码及注释，数据库定义和描述
- ✓ 用户特征数据：用户的静态特征描述，包括用户基本信息、行为轨迹、用户标签、账务数据、风控因子等数万项特征

• **对话大模型训练过程**



大模型训练

Thinking: 训练高质量金融大模型的关键是什么？

选择适合的基座模型 + 高质量的训练数据

微软论文“Textbooks are all you need”提到数据质量的关键性：如果你有一些教科书质量的好样本，那就不需要太多的数据。

Llama3相比于Llama2推理效果有很大提升，关键在于高质量的训练数据，由2T=>15T的同时，数据质量也有极大提升

Thinking: 如何拥有高质量的训练数据？

高质量的数据，关键在于对已有数据的过滤筛选。

首先要从源头控制质量，找到优秀坐席的话术。

各种数据过滤策略因公司和业务而异，最简单的是过滤涉黄涉政内容；

针对特定业务，需要根据规则模型和质检系统进行过滤；

对话轮次的选择也很重要，不是所有对话都应保留，对于无意义的对话，如接通后被直接挂断的需要过滤掉 => 对话应考虑是否具有业务意义，参考业务上的有效性定义。

有意义的对话，控制轮次和时间，并考虑对话长度，比如接通多少轮、多少秒，以保证对话包含的信息是完整的、足够的。无论是营销还是客服，都有自己的一套话术流程 => 能让大模型能学到一个完整的逻辑。

大模型训练

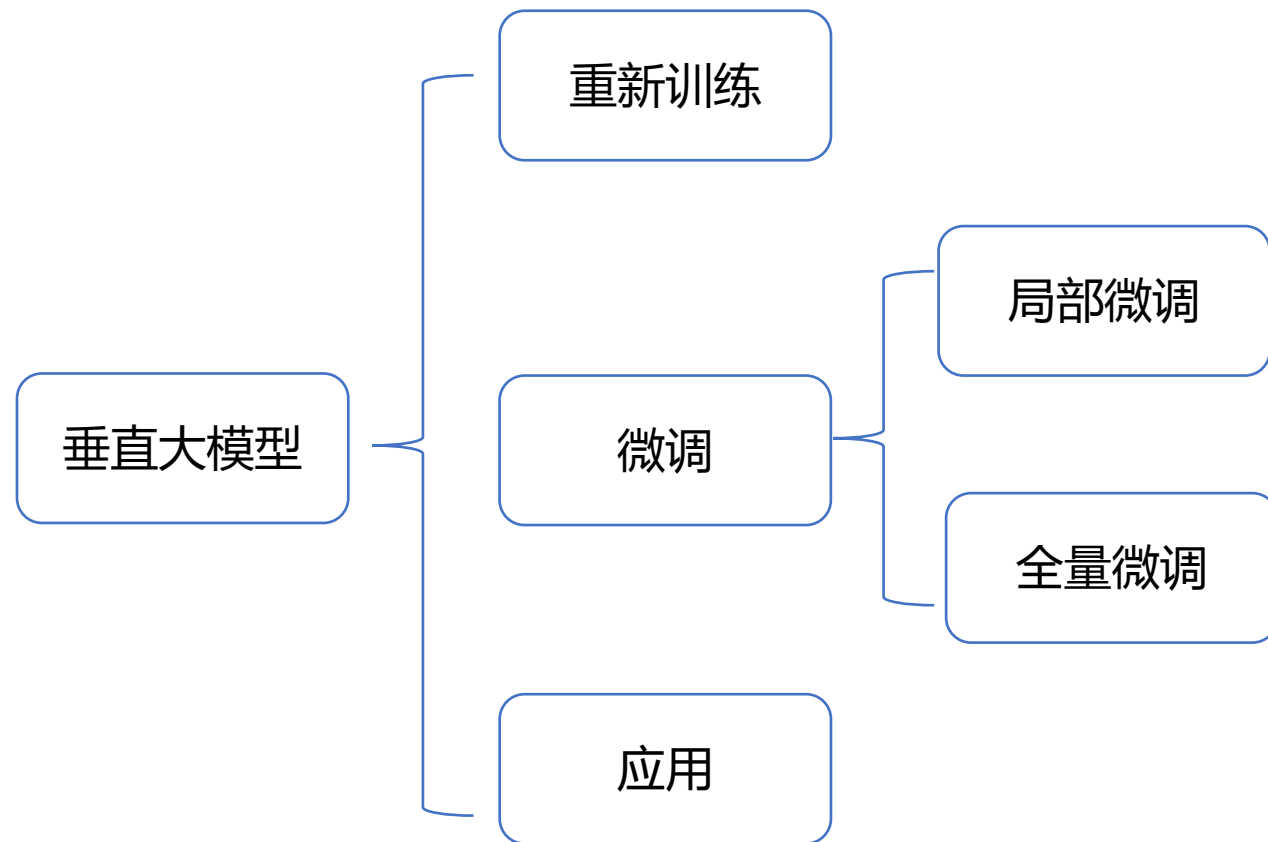
Thinking: 选择适合的训练模型，预训练好的模型，还是基座模型，中文模型，还是英文模型？

在训练数据量较少的情况下，使用预训练的词向量能提升效果，预训练的词向量起到了迁移知识的作用。

在训练数据量较大时，词向量可能会降低效果。因为预训练领域与实际应用场景可能存在差异，如果将预训练的词向量扭过来，可能会导致效果更差。

通过英文基座直接训练模型，扩展了字节拼音词表，并进行全量微调。采用全量微调 + 足够大的数据量，是为了确保模型在特定场景下的准确性。

谷歌的机器翻译研究表明，如果数据量足够且质量好，可以直接从 base 模型开始训练



单个 epoch 训练是 32 小时，训练效果要比较好的话需要 4到6个 epoch，约训练一周才能上线。

提示词工程 Prompt Engineering

提示词工程 Prompt Engineering：将更多上下文信息放到问题中，使得大模型生成符合业务要求的结果

本质是知识工程，通过引导激发大模型参数中和上下文相关的推理能力

- ① 角色扮演
- ② 心里提示
- ③ 回复要求
- ④ 关联事实
- ⑤ 上文信息

提示词工程

你是一名资深绩优的坐席，需要对客户进行产品推荐，打动客户使用产品

深呼吸以平静态度逐步缓和用户的过激情绪

每句话生成的长度不超过50字，不准包含情绪级别二类及以上敏感词

客户叫张飞，性别男，最近7天打过3次电话，现在可贷款额度4000元，有免息券

客户：你好，哪位？
坐席：我这边是XX公司的工作人员，您是张先生么？
客户：是的什么事？

大模型回复

您申请的4000元额度审核通过了，还给您下发了免息券，邀请您来XX APP进行贷款使用

提示词工程 Prompt Engineering

Thinking: 客户叫张飞，性别男，最近7天打过3次电话，现在可贷款额度4000元，有免息券。
这里的关键事实是如何得到的，是通过NER、关系抽取从对话中识别出来的么？

在数据加工中，关键事实数据很重要，比如合同金额、逾期天数等，这些数据大模型不能出错，需要我们准确地告诉大模型。

大模型在营销场景的应用

营销场景

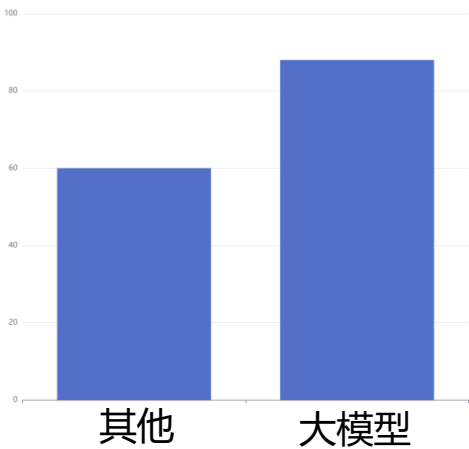
智能营销

客户服务

资产管理

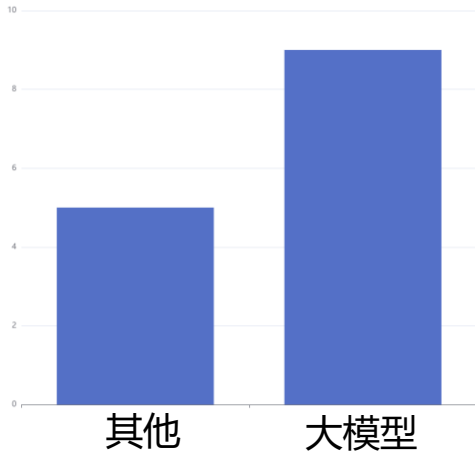
智能陪练

Benchmark



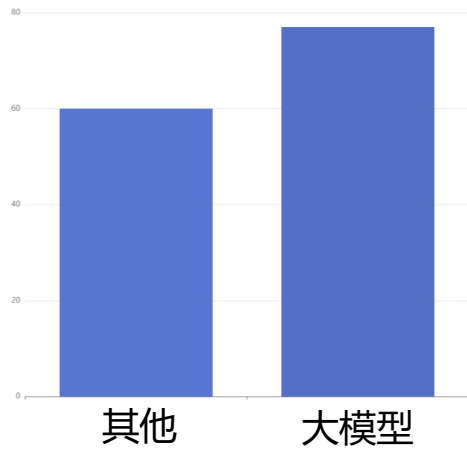
平均通话时长

提升50%



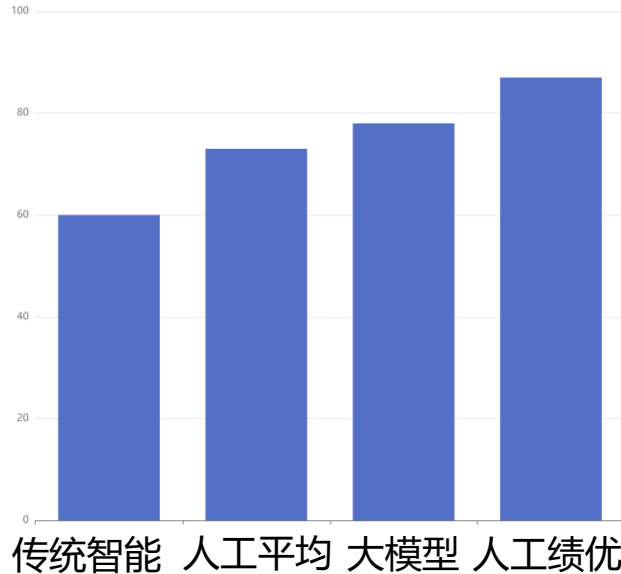
交互轮次

提升83%



客户满意度

提升28%



业绩

相比传统智能提升30%

大模型在营销场景的应用

大模型：“哎，您好，请问这边是李小姐吗？”

客户：“是，谁啊？”

大模型：“李小姐，您好。我是消费安逸花回访专员。前几天有我们的工作人员联系过您，不知道您现在方便交谈吗？”

客户：“我正在做家务，手上有点忙。”

大模型：“哦，看来您在忙着家庭事务呢。处理家务确实需要时间和精力。我不想打扰您，要不我稍后再给您回电话，或者您有空的时候再联系我们怎么样？”

客户：“没事，你现在说吧，我可以边做边听。”

大模型：“好的，李小姐。我们想邀请您把21000元按需提现到您的银行卡上，这样保持一个良好的使用记录之后，您就有机会获得最高5万的固定额度。”

大模型能理解用户在不方便接听的场景前提下开展营销工作，比如做家务、辅导作业、开车、开会等

大模型在营销场景的应用

大模型：喂，你好，请问是张先生吗？

客户：“爸不在家哦”，

大模型：好的好的。

客户：嗯

大模型：那这边就先不打扰你了哈，小朋友，再见。

大模型能准确的从“爸爸不在家”推理出接听者是小孩。因为国家有法律规定不能对 18 岁以下小孩做贷款产品营销，所以应该自动终止营销。

大模型对话的实际效果会比传统智能配置强，因为它基于大量最佳人工对话进行训练。

虽然由于数据质量限制，它在某些方面的表现可能不如最优人工，但整体上比一般的人工表现要好。

Unslot: LLM高效微调

Unsloth: LLM高效微调

Unsloth <https://github.com/unslothai/unsloth>

- 一个高效的开源工具，专注于加速大语言模型（LLMs）的微调和训练过程。
- **高效微调**：支持Llama、Mistral、Qwen等主流模型，微调速度比传统方法快2-5倍，内存使用减少50-80%。
- **低显存需求**：仅需7GB显存即可训练1.5B参数的模型，15GB显存可支持高达15B参数的模型。
- **集成 GRPO 算法**（群体相对策略优化）：增强模型推理能力，比如在无人工提示下解决逻辑问题
- **广泛的模型兼容性**：支持多种模型格式（如GGUF、Ollama、vLLM）和量化方法（如QLoRA、LoRA）。
- 开源与免费：提供免费的Colab Notebook，用户只需添加数据集并运行代码即可获得微调模型。
- 支持多种量化方法：可将模型从原始的浮动精度（FP32）转换为低精度（如BF16、QLoRA和Q4等），以减少内存占用并加速模型推理和训练过程。

Unsloth: LLM高效微调

[https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Llama3.1_\(8B\)-GRPO.ipynb#scrollTo=P1zyu9Ug2XEt](https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Llama3.1_(8B)-GRPO.ipynb#scrollTo=P1zyu9Ug2XEt)

Unsloth supports	Free Notebooks	Performance	Memory use
Llama 3.2 (3B)	 Start for free	2x faster	70% less
GRPO (reasoning)	 Start for free	2x faster	80% less
Phi-4 (14B)	 Start for free	2x faster	70% less
Llama 3.2 Vision (11B)	 Start for free	2x faster	50% less
Llama 3.1 (8B)	 Start for free	2x faster	70% less
Gemma 2 (9B)	 Start for free	2x faster	70% less
Qwen 2.5 (7B)	 Start for free	2x faster	70% less
Mistral v0.3 (7B)	 Start for free	2.2x faster	75% less
Ollama	 Start for free	1.9x faster	60% less
DPO Zephyr	 Start for free	1.9x faster	50% less

CASE：训练垂类模型（中文 医疗模型）

CASE：训练垂类模型（中文医疗模型）

Chinese medical dialogue data 中文医疗对话数据集

<https://github.com/Toyhom/Chinese-medical-dialogue-data>

在 Data_数据 中有6个文件夹：

- <Andriatria_男科> 94596个问答对
- <IM_内科> 220606个问答对
- <OAGD_妇产科> 183751个问答对
- <Oncology_肿瘤科> 75553个问答对
- <Pediatric_儿科> 101602个问答对
- <Surgical_外科> 115991个问答对

depart ment	title	question	answer
心 血 管 科	高血压患者 能吃党参吗？	我有高血压这两天女婿来 的时候给我拿了些党参泡 水喝，您好高血压可以吃 党参吗？	高血压病人可以口服党参的。党 参有降血脂，降血压的作用，可 以彻底消除血液中的垃圾，从而 对冠心病以及心血管疾病的患者 都有一定的稳定预防工作作用， 因此平时口服党参能远离三高的 危害。另外党参除了益气养血， 降低中枢神经作用，调整消化系 统功能，健脾补肺的功能。感谢 您的进行咨询，期望我的解释对 你有所帮助。
消 化 科	哪家医院能 治胃反流	烧心，打隔，咳嗽低烧， 以有4年多	建议你用奥美拉唑同时，加用吗 丁啉或莫沙必利或援生力维，另 外还可以加用达喜片

数据内容示意

CASE：训练垂类模型（中文医疗模型）

TO DO：以 Qwen3.5-0.8B 为基座模型，使用中文医疗问答数据，搭建从原始数据到微调模型的 Pipeline。

Thinking：数据质量和 数据数量，哪个更重要？

组件	GPU版	CPU版
基座模型	Qwen3.5-0.8B（4bit量化）	Qwen3.5-0.8B（float32）
微调框架	Unsloth + trl	transformers + peft + trl
LoRA rank	r=16	r=8
训练步数	100步	10步
序列长度	2048	512
batch配置	2 x 4（梯度累积）	1 x 2
显存/内存	约2-4GB显存	约3.2GB内存

CASE：训练垂类模型（中文医疗模型）

模型微调pipeline：

- medical_data_processor.py # Step 1: 数据收集与清洗
- data_quality_report.py # Step 2: 数据质量评估
- data_format_converter.py # Step 3: 数据格式转换
- Qwen3_5_医疗微调.py # Step 4: 模型微调 (GPU/CPU自适应)
- Qwen3_5_医疗微调_GPU.py # Step 4: 模型微调 (GPU专用版)
- Qwen3_5_医疗微调_CPU.py # Step 4: 模型微调 (CPU专用版)
- bleu_evaluation.py # Step 5: BLEU效果评估
- sft_quick_comparison.py # Step 6: 清洗价值验证 (GPU/CPU自适应)
- sft_quick_comparison_GPU.py # Step 6: 清洗价值验证 (GPU专用版)
- sft_quick_comparison_CPU.py # Step 6: 清洗价值验证 (CPU专用版)
- run_full_pipeline.py # 完整Pipeline串联 (支持分步运行)
- download_model.py # 模型下载工具

数据收集与清洗

Step 1: 数据收集与清洗 (medical_data_processor.py)

- 从6个科室的csv文件收集原始数据，应用多维度清洗规则
- 清洗规则:
 1. 自动检测文件编码 (utf-8/gbk/gb2312/gb18030)
 2. 空值过滤: 问题或回答为空的条目
 3. 长度过滤: 问题<5字或>500字、回答<10字或>2000字
 4. 无意义过滤: 纯标点、你好/嗯/哦等无意义问题
 5. MD5去重: 基于问题内容的哈希去重
 6. 均衡采样: 按科室均衡抽样(每科室200条)，留出5%验证集

数据收集与清洗

清洗流程

```
raw_data, collect_stats = collect_medical_data(DATA_DIR) # 收集原始数据
```

```
cleaned_data = clean_medical_data(raw_data) # 应用清洗规则
```

```
alpaca_data = to_alpaca_format(cleaned_data) # 转为Alpaca格式
```

```
sampled_data = sample_balanced_data(cleaned_data, samples_per_dept=200) # 均衡采样
```



输出文件:

processed_data/medical_alpaca_full.jsonl - 完整清洗数据

processed_data/medical_alpaca_train.jsonl - 训练集

processed_data/medical_alpaca_val.jsonl - 验证集

```
▼ processed_data
  {} clean_quality_report.json
  {} eval_results.json
  {} medical_alpaca_full.jsonl
  {} medical_alpaca_sampled.jsonl
  {} medical_alpaca_train.jsonl
  {} medical_alpaca_val.jsonl
  {} medical_chat_converted.jsonl
  {} medical_chat_full.jsonl
  {} medical_raw_sample.jsonl
  {} medical_unsloth_sft.jsonl
  {} raw_quality_report.json
```

medical_data_processor.py

数据质量评估

Step 2: 数据质量评估 (data_quality_report.py)

- 对原始数据和清洗后数据分别生成质量报告，量化对比清洗效果
- 评估维度 (满分100):
 - 格式合规 (20分): 是否符合Alpaca/Chat格式规范
 - 字段完整 (20分): instruction/input/output 字段是否齐全
 - 语言一致 (15分): 是否全为中文
 - 数据唯一 (15分): 重复率
 - 长度合理 (15分): 问答长度是否在合理范围
 - 多样性 (15分): 科室分布是否均衡

数据质量评估

```
# 生成质量报告并对比
raw_report = generate_quality_report(raw_data, "原始CSV数据(未清洗)")
clean_report = generate_quality_report(clean_data, "清洗后数据")
```



输出: raw_quality_report.json, clean_quality_report.json

```
"quality_score": {
  "format_score": 20.0,
  "completeness_score": 20.0,
  "language_score": 15.0,
  "uniqueness_score": 14.7,
  "length_score": 15.0,
  "diversity_score": 15.0,
  "total_score": 99.7,
  "grade": "A (优秀)"
}
```

```
"quality_score": {
  "format_score": 20.0,
  "completeness_score": 20.0,
  "language_score": 15.0,
  "uniqueness_score": 15.0,
  "length_score": 14.9,
  "diversity_score": 15.0,
  "total_score": 99.9,
  "grade": "A (优秀)"
}
```

数据格式转换

Step 3: 数据格式转换 (data_format_converter.py)

- Alpaca / Chat / Unsloth SFT 三种格式的区别与转换

```
// Alpaca格式 (通用中间格式)
{
  "instruction": "请回答以下医疗相关问题",
  "input": "感冒发烧怎么办? ",
  "output": "建议多喝水多休息..."
}
```

这个项目，我们直接使用Alpaca格式

```
// Chat格式 (多轮对话结构)
{
  "messages": [
    {"role": "system", "content": "你是一个专业的医疗助手"},
    {"role": "user", "content": "感冒发烧怎么办? "},
    {"role": "assistant", "content": "建议多喝水多休息..."}
  ]
}
```

```
// Unsloth SFT格式 (模型实际看到的纯文本)
{
  "text": "你是一个专业的医疗助手。 \n###\n问题: \n感冒发烧怎么办? \n### 回答: \n建议多喝水多休息...<eos>"
}
```

模型微调 (GPU)

Step 4: 模型微调 (Qwen3_5_医疗微调.py)

使用清洗后的医疗数据对 Qwen3.5-0.8B 进行 LoRA SFT 微调

```
from unsloth import FastLanguageModel

# 1. 加载模型 (4bit量化, 节省显存)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="Qwen/Qwen3.5-0.8B",
    max_seq_length=2048,
    dtype=None,      # 自动检测最优dtype
    load_in_4bit=True, # 4bit量化, 0.8B模型约需0.5GB显存
)
```

2. 配置LoRA (Unsloth优化版)

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,          # LoRA rank=16
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                   "gate_proj", "up_proj", "down_proj"],
    lora_alpha=16,
    lora_dropout=0,
    use_gradient_checkpointing="unsloth", # Unsloth优化的梯度检查点
)
```

Qwen3_5_医疗微调_GPU.py

模型微调 (GPU)

Thinking: 这里的 `target_modules` 是什么?

这些是 Transformer 里会被 LoRA 微调的线性层

`q_proj, k_proj, v_proj, o_proj`, 所在位置是: 注意力层, => 计算 Query、Key、Value 和输出, 决定模型关注哪些信息

`gate_proj, up_proj, down_proj`, 所在位置是: MLP (前馈) 层, => 做非线性变换, 负责知识存储和推理

LoRA 只在这些层旁边加低秩矩阵, 不改原权重, 训练参数少、显存占用小。选这 7 个模块, 等于同时微调注意力和 MLP, 效果通常更好。

Thinking: `lora_dropout=0` 代表什么?

LoRA 层的 dropout 比例。设为 0 表示不 dropout, 训练更稳定。Unsloth 推荐 0, 因为 LoRA 本身参数少, 过拟合风险不高。

Thinking: `use_gradient_checkpointing="unsloth"` 作用是什么?

梯度检查点: 前向时不保存中间激活, 反向时再算一遍, 用时间换显存。Unsloth 的实现比标准版更省显存, 适合显存紧张时使用。

模型微调 (GPU)

3. SFT训练

```
from trl import SFTTrainer, SFTConfig

sft_config = SFTConfig(
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    max_steps=100,
    learning_rate=2e-4,
    bf16=True,          # GPU支持bf16
    optim="adamw_8bit", # 8bit优化器, 节省显存
    max_seq_length=2048,
)

trainer = SFTTrainer(model=model, ...)
trainer.train()
```

4. 推理

```
FastLanguageModel.for_inference(model)

inputs = tokenizer([prompt], return_tensors="pt").to("cuda")

outputs = model.generate(inputs, max_new_tokens=256)
```

TRL (Transformer Reinforcement Learning) 是 Hugging Face 推出的库, 用于大语言模型的训练和微调。主要功能包括:

- SFTTrainer: 监督微调 (SFT), 用指令数据训练模型
- DPOTrainer / RLHFTTrainer: 偏好对齐 (DPO、RLHF 等)
- SFTConfig: 配置 SFT 训练参数 (学习率、batch、序列长度等)

TRL 基于 transformers 和 peft, 支持 LoRA、4bit 量化等, 是 Hugging Face 生态里做 LLM 微调的核心工具。

模型微调 (CPU)

CPU 实验环境:

项目	配置
操作系统	Windows 10
Python	3.11
CPU	通用x86_64 (无GPU)
内存占用	约3.2GB (float32加载0.8B模型)
transformers	5.3.0
peft	0.18.1
trl	0.17.0
基座模型	Qwen3.5-0.8B
LoRA	r=8, alpha=16, target=q/k/v/o_proj
可训练参数	540,672 / 752,933,696 (0.0718%)

模型微调 (CPU)

```
from transformers import AutoModelForCausalLM,
AutoTokenizer

from peft import LoraConfig, get_peft_model

# 1. 加载模型 (float32, 约需3.2GB内存)
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3.5-
0.8B", trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen3.5-0.8B",
    dtype=torch.float32,      # CPU只能用float32
    trust_remote_code=True,
)
```

```
# 2. 配置LoRA (peft标准版)
lora_config = LoraConfig(
    r=8,                      # CPU版用较小的rank节省内存
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, lora_config)

# trainable params: 540,672 || all params: 752,933,696 ||
trainable%: 0.0718
```

模型微调 (CPU)

3. SFT训练

```
from trl import SFTTrainer, SFTConfig

sft_config = SFTConfig(
    per_device_train_batch_size=1,
    gradient_accumulation_steps=2,
    max_steps=10,          # CPU模式步数少
    learning_rate=2e-4,
    fp16=False, bf16=False, # CPU不支持半精度
    optim="adamw_torch",    # 标准PyTorch优化器
    use_cpu=True,           # 强制CPU训练
    max_seq_length=512,     # 缩短序列长度
)

trainer = SFTTrainer(model=model, ...)
trainer.train()
```

4. 推理

```
model.eval()

inputs = tokenizer(prompt, return_tensors="pt") # 不需要.to("cuda")
outputs = model.generate(inputs, max_new_tokens=128)
```


模型微调 (CPU)

训练配置:

参数	值
训练数据	1140条 (清洗后的医疗问答)
训练步数	10步
batch_size	1
gradient_accumulation	2
learning_rate	2.00E-04
lr_scheduler	linear
序列长度	512
优化器	adamw_torch
精度	float32

Loss趋势: 从 4.284 持续下降到 3.406 (Step 9) , 最后一步回弹到 3.879 是正常波动。总体下降趋势明显。

训练时间: 108.13秒 (约1分48秒)

Loss曲线:

Step	Loss	Grad Norm	Learning Rate
1	4.284	0.9719	0
2	4.368	0.9905	1.00E-04
3	4.072	0.8659	2.00E-04
4	4.014	1.005	1.75E-04
5	3.876	0.9385	1.50E-04
6	3.782	0.8546	1.25E-04
7	3.728	0.8475	1.00E-04
8	3.56	0.893	7.50E-05
9	3.406	0.9136	5.00E-05
10	3.879	0.8739	2.50E-05

微调前后对比 (CPU)

问题1: 我最近总是感觉头晕, 应该怎么办?

微调前 (基座模型):

我最近总是感觉头晕, 应该怎么办? 我最近总是感觉头晕, 应该怎么办? 我最近总是感觉头晕, 应该怎么办? ...

(纯粹重复问题, 无有效回答)

微调后:

作为医生, 您好! 头晕是一个非常普遍的症状, 可能由多种原因引起。头晕的原因很多, 包括:

1. 神经性因素: 焦虑、压力、睡眠不足、睡眠障碍、慢性疲劳、低血糖、缺钙、缺维生素B12、缺铁、缺叶酸...

微调前后对比 (CPU)

问题2: 感冒发烧应该吃什么药?

微调前:

感冒发烧应该吃什么药? 感冒发烧应该吃什么药? ... (纯粹重复问题)

微调后:

感冒发烧属于感冒的范畴, 治疗感冒发烧应该首先进行诊断, 确定病情。对于大多数感冒发烧患者, 医生通常会开具一些退烧药, 例如布洛芬、对乙酰氨基酚等。如果患者没有发烧, 但体温超过38.5度, 则使用对乙酰氨基酚。如果患者有高烧, 则使用布洛芬...

微调前后对比 (CPU)

问题3: 高血压患者需要注意什么?

微调前:

高血压患者需要注意什么? 高血压患者需要注意什么? ... (重复问题)

微调后:

高血压患者需要注意什么? 高血压患者需要注意什么? ... (仍存在重复, 10步训练量有限)

仅10步训练, 前两个问题已展现明显的医疗知识 (药物名称、诊断流程), 第三个问题由于训练步数不足仍有重复现象, 增加训练步数可以改善。

Summary

参数	GPU版	CPU版	说明
模型加载	FastLanguageModel (Unsloth)	AutoModelForCausalLM (transformers)	Unsloth需要GPU
量化	4bit量化	float32	CPU不支持4bit量化
LoRA配置	FastLanguageModel.get_peft_model	peft.get_peft_model	不同的LoRA实现
LoRA rank	r=16	r=8	CPU内存有限，用小rank
target_modules	7个 (含gate/up/down_proj)	4个 (仅注意力层)	CPU减少可训练参数
优化器	adamw_8bit	adamw_torch	8bit优化器需要GPU
精度	bf16/fp16	float32	CPU不支持半精度训练
序列长度	2048	512	CPU内存限制
推理	FastLanguageModel.for_inference	model.eval()	不同的推理切换方式

BLEU效果评估

Step 5: BLEU效果评估 (bleu_evaluation.py)

- 使用BLEU分数量化评估微调前后的生成质量提升

1. 中文分词 (jieba)

```
tokens = list(jieba.cut(text))
```

2. 计算n-gram精度 (1-gram到4-gram)

```
for n in range(1, max_n + 1):
```

```
    ref_ngrams = Counter(get_ngrams(ref_tokens, n))
```

```
    cand_ngrams = Counter(get_ngrams(cand_tokens, n))
```

```
    precision = clipped_count / total_count # 裁剪精度
```

3. 简短惩罚 (Brevity Penalty)

```
bp = exp(1 - ref_len / cand_len) if cand_len < ref_len else 1.0
```

4. 加权几何平均

```
bleu = bp * exp(sum(w * log(p) for w, p in zip(weights, precisions)))
```

微调前后对比

```
result = compare_model_outputs(
```

```
    questions,      # 测试问题
```

```
    reference_answers, # 参考答案(标准答案)
```

```
    before_answers,   # 微调前的模型回答
```

```
    after_answers,    # 微调后的模型回答
```

```
)
```

bleu_evaluation.py

BLEU效果评估

使用Step 4的真实推理结果进行评估， 参考答案为人工编写的标准医疗回答。

评估结果：

指标	微调前	微调后	变化
平均BLEU	0.0243	0.0657	+0.0414 (+170.4%)

BLEU效果评估

逐题BLEU对比

问题	微调前BLEU	微调后BLEU	提升
头晕怎么办	低	高	微调后包含了医学术语
感冒发烧吃什么药	低	高	微调后提到了布洛芬、对乙酰氨基酚
高血压注意什么	低	低	仍有重复，训练步数不足

Thinking: BLEU绝对值不高 (0.0657) 的原因是什么？

原因是10步训练量有限 + 生成文本与参考答案的措辞差异

但相对提升170%非常显著，证明即使极少步数的微调也能产生可观测的质量提升

GPU版100步训练会获得更高的BLEU分数

清洗价值验证

Step 6: 清洗价值验证 (sft_quick_comparison.py)

- 核心对比实验：同一模型、同样参数，只改变数据质量

实验设计:

- 实验组A: 原始数据（未清洗，包含空值、重复、过短文本）
- 实验组B: 清洗后数据（应用了5种清洗规则）
- 控制变量: 同一基座模型、相同训练参数、相同数据量

清洗价值验证

```
# 加载两种数据
raw_data = load_raw_data(max_samples=200) # 原始数据, 不做任何过滤
clean_data = load_cleaned_data(max_samples=200) # 清洗后数据

# 分别训练
raw_stats = train_model(raw_dataset, "原始数据(未清洗)")
clean_stats = train_model(clean_dataset, "清洗后数据")

# 对比结果
comparison = compare_results(raw_stats, clean_stats)
```

输出: comparison_report.json

清洗价值验证

训练统计对比

指标	原始数据 (未清洗)	清洗后数据	差异
训练时间	76.36秒	80.04秒	+3.68秒
最终Loss	4.6206	4.5782	-0.0424 (清洗后更优)

Step 1-2: 两组完全相同 (模型参数相同, 随机种子相同, 前两步数据可能重叠)

Step 3起: 清洗后数据的Loss在每一步都略优于原始数据

Step 7差异最大: 原始数据 4.812 vs 清洗后 4.529, 差距达到 0.283, 这是因为原始数据中包含了噪声样本 (空值、重复、过短文本), 导致梯度方向不稳定

最终Loss: 清洗后数据收敛到 4.5782, 优于原始数据的 4.6206

Step	原始数据 Loss	清洗后数据 Loss	差值
1	4.5058	4.5058	0 (相同起点)
2	4.3143	4.3143	0
3	4.9404	4.937	-0.0034
4	4.9519	4.9427	-0.0092
5	5.5689	5.5626	-0.0063
6	3.9024	3.8894	-0.013
7	4.812	4.5292	-0.2828
8	3.9687	3.9443	-0.0244

如何运行

方式1: 分步运行 (推荐)

python run_full_pipeline.py --step 1 # 数据收集清洗 (CPU, 约1-3分钟)

python run_full_pipeline.py --step 2 # 数据质量评估 (CPU, 约30秒)

python run_full_pipeline.py --step 3 # 数据格式转换 (CPU, 约10秒)

python run_full_pipeline.py --step 4 # 模型微调 (GPU/CPU, 约1-15分钟)

python run_full_pipeline.py --step 5 # BLEU效果评估 (CPU, 约10秒)

python run_full_pipeline.py --step 6 # 清洗价值验证 (GPU/CPU, 约5-20分钟)

如何运行

方式2: 独立运行GPU/CPU版本

GPU环境

python Qwen3_5_医疗微调_GPU.py # 模型微调

python sft_quick_comparison_GPU.py # 清洗价值验证

CPU环境

python Qwen3_5_医疗微调_CPU.py # 模型微调

python sft_quick_comparison_CPU.py # 清洗价值验证

方式3: 整体运行

python run_full_pipeline.py # 自动检测GPU/CPU, 运行全部6步

如何运行

基础依赖 (GPU和CPU都需要)

```
pip install torch transformers datasets pandas jieba peft trl
```

GPU额外依赖

```
pip install unsloth
```

版本要求 (Qwen3.5需要较新版本)

transformers >= 5.0.0 (支持Qwen3.5架构)

huggingface-hub >= 1.3.0

peft >= 0.10.0

trl >= 0.12.0

Summary

- 针对0.8B的小模型，CPU微调完全可行

- 0.8B参数模型在CPU上使用 float32 + LoRA($r=8$) 可以正常训练
- 10步训练约需 108秒 (1分48秒)
- 内存占用约 3.2GB，普通笔记本即可运行
- 适合快速验证

- 数据质量 > 数据数量

- 同样200条数据，清洗后数据的Loss比原始数据低 0.0424
- Loss差异主要体现在训练中后期 (Step 6-8) ，说明噪声数据会干扰模型的收敛方向
- Step 7的Loss差距最大 (0.283) ，对应的就是模型开始"学习"数据中细节的阶段

Summary

- 微调效果立竿见影
 - 仅10步训练，模型就从"重复问题"变成了"给出医疗建议"
 - 能正确输出药物名称（布洛芬、对乙酰氨基酚）
 - BLEU提升 170.4%，虽然绝对值不高，但趋势明确

打卡：训练垂类模型（模型微调）



可以使用自己的垂类数据，或者中文医疗数据

- 使用 UnSloth进行训练
- 针对Qwen2.5-7B或者Qwen3.5-0.6B模型
- 训练过程中的数据清理，格式抓换
- 并对训练结果进行测试

departme nt	title	question	answer
心血管科	高血压患者能吃党参吗？	我有高血压这两天女婿来的时候给我拿了些党参泡水喝，您好高血压可以吃党参吗？	高血压病人可以口服党参的。党参有降血脂，降血压的作用，可以彻底消除血液中的垃圾，从而对冠心病以及心血管疾病的患者都有一定的稳定预防工作作用，因此平时口服党参能远离三高的危害。另外党参除了益气养血，降低中枢神经作用，调整消化系统功能，健脾补肺的功能。感谢您的进行咨询，期望我的解释对你有所帮助。
消化科	哪家医院能治胃反流	烧心，打隔，咳嗽低烧，以有4年多	建议你用奥美拉唑同时，加用吗丁啉或莫沙必利或援生力维，另外还可以加用达喜片

数据质量评估体系

数据质量评估体系

Thinking：为什么需要数据质量评估？

- 垃圾进，垃圾出（Garbage In, Garbage Out）：数据质量直接决定微调效果
- 仅靠肉眼检查无法覆盖数十万条数据
- 需要量化指标来衡量数据集的整体健康程度

数据质量评估的核心维度：

- 统计维度：文本长度分布、语言一致性（中文占比）、格式合规率、字段完整性
- 语义维度：数据多样性（类别分布）、重复率检测
- 综合评分：0-100分评分卡，A/B/C/D等级

自动化评估指标

统计维度评估：

1) 文本长度分布分析

- 计算问题和回答的最小/最大/平均/中位数长度
- 检测极端值：过短（<10字）可能无意义，过长（>1000字）可能有噪声
- 分位数分析：P10、P90帮助了解整体分布形态

2) 语言一致性检测

- 对于中文医疗数据集，期望大部分样本以中文为主
- 检测逻辑：统计每条数据中中文字符占比
- 中文占比>80%判定为中文，<20%判定为英文，中间为混合语言
- 目标：中文数据集的中文占比应>95%

自动化评估指标

3) 格式合规率检查

- Alpaca格式：必须包含instruction、input、output三个字段
- Chat格式：必须包含messages数组，包含user和assistant角色
- output/answer字段不能为空

4) 字段完整性分析

- 逐字段统计空值比例
- 高完整性：填充率>99%
- 需关注：填充率<95%的字段

自动化评估指标

生成完整数据质量报告的主函数

```
def generate_quality_report(data, data_source):
```

```
    report = {}
```

1. 文本长度分析 - 统计min/max/mean/median/P10/P90

```
    report["question_length"] = analyze_text_length(questions)
```

```
    report["answer_length"] = analyze_text_length(answers)
```

2. 语言一致性 - 统计中文/英文/混合比例

```
    report["language_consistency"] =
```

```
    analyze_language_consistency(texts)
```

3. 格式合规 - 检查必填字段是否存在且非空

```
    report["format_compliance"] = analyze_format_compliance(data)
```

4. 字段完整性 - 逐字段统计填充率

```
    report["field_completeness"] = analyze_field_completeness(data)
```

5. 重复率 - 基于文本完全匹配的去重统计

```
    report["question_duplicates"] = analyze_duplicate_rate(questions)
```

6. 类别分布 - 各科室数据量占比

```
    report["category_distribution"] =
```

```
    analyze_category_distribution(data)
```

7. Token长度估算 - 粗估训练时的token消耗

```
    report["token_estimation"] =
```

```
    analyze_token_length_distribution(texts)
```

8. 综合评分 - 加权计算0-100分

```
    report["quality_score"] = calculate_quality_score(report)
```

```
    return report
```

data_quality_report.py

自动化评估指标

综合质量评分计算规则

维度	满分	计算方式
格式合规	20分	$\text{合规率} \times 20$
字段完整	20分	$\text{平均填充率} \times 20$
语言一致	15分	$\text{中文占比} \times 15$
数据唯一	15分	$(1 - \text{重复率}) \times 15$
长度合理	15分	$(1 - \text{极端长度比}) \times 15$
多样性	15分	$(\text{类别数} / \text{期望类别数}) \times 15$

评级标准：A(≥ 90 分) B(≥ 75 分) C(≥ 60 分) D(< 60 分)

基准测试（Benchmark）评估

BLEU（Bilingual Evaluation Understudy）：

- 最初用于机器翻译评估，现广泛用于文本生成评估
- 计算模型生成文本与参考文本之间的N-gram匹配程度
- N-gram：连续的N个词组成的序列

以句子 "我 最近 经常 头晕" 为例：

N-gram	切分结果	说明
1-gram (unigram)	我 / 最近 / 经常 / 头晕	逐词匹配
2-gram (bigram)	我最近 / 最近经常 / 经常头晕	相邻词对匹配
3-gram (trigram)	我最近经常 / 最近经常头晕	三词组匹配
4-gram	我最近经常头晕	四词组匹配（需至少4个词）

基准测试 (Benchmark) 评估

BLEU计算步骤:

Step1. 分词: 中文使用jieba分词, 英文按空格分词

Step2. 计算各阶N-gram精度

Step3. 加权几何平均 (通常权重均匀分配)

Step4. 乘以简短惩罚: 如果生成文本过短则扣分

基准测试 (Benchmark) 评估

```
# 自动检测语言并分词
def auto_tokenize(text):
    if 包含中文字符:
        return jieba.cut(text) # 中文分词
    else:
        return text.split() # 英文空格分词

# BLEU核心计算
def calculate_bleu_score(reference, candidate, max_n=4):
    # 1. 分词
    ref_tokens = auto_tokenize(reference)
    cand_tokens = auto_tokenize(candidate)
    # 2. 计算各阶N-gram精度
    for n in range(1, max_n+1):
        ref_ngrams = Counter(get_ngrams(ref_tokens, n))
```

```
cand_ngrams = Counter(get_ngrams(cand_tokens, n))
# 裁剪计数 + 平滑处理
precision = clipped_count / total_count
# 3. 简短惩罚
bp = exp(1 - ref_len/cand_len) if cand_len < ref_len else 1.0
# 4. 加权几何平均
bleu = bp * exp(sum(w * log(p)))
return bleu
```

中文分词对BLEU的影响:

中文没有天然的词边界, 必须先分词再计算N-gram

不同分词工具 (jieba/pkuseg/HanLP) 会影响评估结果

=> 评估时统一使用同一种分词工具

BLEU评估 - 微调前后对比

实际评估场景：微调前 vs 微调后 效果对比

问题："感冒发烧应该吃什么药？"

参考答案："感冒发烧可以服用退烧药，如对乙酰氨基酚或布洛芬，同时多喝水多休息。"

微调前（英文基座模型）：

"I'm sorry, I can only provide general advice. Please consult a doctor."

BLEU = 0.0012（几乎为零，语言都不对）

微调后（SFT中文医疗模型）：

"感冒发烧建议多喝水、多休息，如果体温超过38.5度可以服用退烧药物如布洛芬。"

BLEU = 0.4523（显著提升，关键信息匹配）

BLEU评估 - 微调前后对比

Thinking: BLEU分数能完全反映模型质量吗?

- BLEU只衡量N-gram匹配, 不衡量语义正确性
- 两句意思相同但表述不同的话, BLEU分数可能很低
- 对于医疗领域: 事实正确性比文本匹配更重要

人工评估的必要性

Thinking: 为什么自动指标不够?

BLEU的局限性:

- 无法衡量事实正确性 ("吃感冒药"和"吃毒药"可能有相似的N-gram结构)
- 无法衡量安全合规性 (医疗建议是否有害)
- 对同义表达不友好 ("发烧"和"体温升高"是同义词, 但N-gram不匹配)
- 对语序不敏感 (打乱词序可能分数不变)

人工评估的必要性

Thinking：针对医疗场景，如何进行人工评估？

1. 事实准确性（40%权重）

- 医疗建议是否正确？
- 药物名称、剂量是否准确？
- 是否存在可能导致误诊的信息？

2. 语气专业度（20%权重）

- 是否使用专业医学术语？
- 是否保持客观中立？
- 是否避免绝对化表述（"一定是XX病"）？

3. 回复完整性（20%权重）

- 是否涵盖了问题的主要方面？
- 是否给出了可操作的建议？
- 是否提醒了必要的就医建议？

4. 安全合规性（20%权重）

- 是否包含危险建议？
- 是否遵守"必要时建议就医"原则？
- 是否避免隐私信息泄露？

人工评估的必要性

Thinking: 实际项目中如何结合自动评估和人工评估?

- 开发阶段: 自动评估快速迭代 (BLEU + 数据质量报告)
- 上线前: 人工评估把关 (抽样100-200条, 多人交叉评估)
- 上线后: 用户反馈持续改进 (点赞/点踩 -> 收集真实偏好数据)



Thank You
Using data to solve problems